Lab 8 — Hibernate / ORM

Gal Bareket 326144300
Itai Aviad 214891665

Task 1 — ORM

Role of ORM. It maps Java classes to database tables. Instead of writing SQL by hand, we annotate our classes (`@Entity`, `@Id`, `@ManyToOne`...) and Hibernate generates the SQL, turns rows into objects and back, and manages the relations (foreign keys, join tables) and transactions for us.

When to prefer plain SQL.

- Complex / performance-critical queries that need hand-tuned SQL.
- Bulk updates/deletes (one statement vs. loading every object).
- Database-specific features (stored procedures, index hints).

Non-built-in annotation. Built-in ones come with Java (`@Override`, `@Deprecated`). The JPA annotations we use come from an external library, e.g. `@Entity` / `@Table`:

```
@Entity
@Table(name = "students")
public class Student { ... }
```

Task 2 — Relationship types

One-to-one — one record links to exactly one on the other side. Example: a `Student` and their single `IdCard`.

```
@OneToOne
@JoinColumn(name = "id_card_id")
private IdCard idCard;
```

One-to-many / many-to-one — one record links to many. Example from this lab: a `Lecturer` leads many `Courses`, each course has one lecturer.

```
// Lecturer
@OneToMany(mappedBy = "lecturer")
private Set<Course> courses;

// Course
@ManyToOne
@JoinColumn(name = "lecturer_id")
private Lecturer lecturer;
```

Many-to-many — each side links to many. It is implemented with a third *join table* holding pairs of foreign keys. Example: a `Student` takes many `Courses` and a course holds many students.

```
// Student (owns the relation)
@ManyToMany
@JoinTable(name = "student_course",
    joinColumns = @JoinColumn(name = "student_id"),
    inverseJoinColumns = @JoinColumn(name = "course_id"))
private Set<Course> courses;

// Course
```

```
@ManyToMany(mappedBy = "courses")
private Set<Student> students;
```

Hibernate creates the `student_course` table automatically (columns `student_id`, `course_id`); each enrollment is one row.